

Service & Environment Failure Strategy — Test Report

Runtime validation of scoped on-failure support and the service-scope RETRY crash fix, run against a live ci-manager instance.

Branch CDS-130614-unified-failure-strategy-service-env

Module 332-ci-manager

Date 2026-09-06

28 TOTAL TESTS ACROSS ALL BATCHES	11 THIS SESSION (E-I)	10 PASSED OUTRIGHT	1 RESOLVED VIA RE- TEST (G2 → G2B)	2 DROPPED, BAD TEST DESIGN
---	---------------------------------	------------------------------	---	---

01 Root cause & fix

Unified (v1) CD only understood a failure rule at the **stage** level. NG (v0) also honors a rule set directly on the **service:** or **environment:** block, and that rule outranks the stage's own — v1 had no equivalent. Wiring it up (UnifiedFailureStrategyHelperV1) surfaced a second, sharper bug: pointing **action: retry** at **service** scope crashed the engine with `Node Retry is supported only for Leaf Nodes`, because v1's Service plan node is a composite (CHILDREN-facilitated) node, not a leaf, and only carries the one recognized escape hatch — a `STEP_GROUP` group tag — when that tag is actually set. v0's own Service node sets it; v1's did not.

```
332-ci-manager/.../plancreator/V1/UnifiedStageServicePlanCreatorUtils.java - createServicePlanNode
PlanNode.builder()
    .uuid(nodeUuid)
    .name(name)
    .group(StepCategory.STEP_GROUP.name())
    .build();
```

Environment-scope retry was never at risk — that node runs under `ASYNC`, which isn't in the engine's `PARENT_MODES` set, so the leaf-only restriction never applied there. Build-verified with `make build t=332-ci-manager`; every test below is the runtime check on top of that build.

02 Prior batches (A–D) — recap

Run and closed out in an earlier session, before this report's E–I batch. Per-test detail wasn't retained beyond this summary line from the tracking doc.

BATCH	COUNT	RESULT	NOTE
A1–A8	8	PASS	Baseline schema/plan-creation checks for scoped on–failure parsing.
B1–B2	2	PASS	Service/environment scope priority over the stage rule.
C1–C2	2	PASS	—
C3	1	DROPPED	Bad test design (entity-existence validation ran before the failure-strategy code did) — not a product bug.
D1–D5	5	PASS	—

03 This session — categories E–I

E1	Service scope retry exhausted → abort	PASS
CHECKING	That retry actually retries the configured number of times at service scope, then falls through to the allowed failure-action once exhausted.	
PREDICTED	Retries the configured attempts, then final status Aborted .	
OUTPUT	Confirms the <code>.group(STEP_GROUP)</code> fix works — no crash. Service node went straight Queued → Aborted (retries against the fake id fail fast, so the UI shows no intermediate retry state), matching the prediction.	
E2	Environment scope retry exhausted → fail	PASS
CHECKING	Same as E1, at environment scope.	
PREDICTED	Retries the configured attempts, then final status Failed .	
OUTPUT	First attempt used a service id that didn't exist and masked the test — re-run with a real service: Service succeeded, Infrastructure failed on the fake infra id, retries exhausted, final status Failed. Confirms environment-scope retry works end-to-end.	
E3	Service retry+abort beats the stage's own fail rule	PASS
CHECKING	Scope priority holds even through the retry/fallback-action resolution path, not just the first check.	
PREDICTED	Retries once, then ends Aborted — stage's fail rule never engages.	
OUTPUT	Service node retried then landed on Aborted; final pipeline status Aborted. Stage's fail rule never engaged.	

F1	Service parallel items, group abort	DROPPED
CHECKING	On-failure applies uniformly to a multi-item/parallel service block, not just single items.	
OUTPUT	Not re-run — same bad test-design flaw as C3 (entity-existence validation short-circuits before the failure-strategy path). No product signal either way.	

F2	Environment parallel items, group abort	PASS (MOSTLY)
CHECKING	Multi-environment/group failures are covered by the group-level rule, not just a single-environment shape.	
PREDICTED	Both items fail, group's abort fires, stage ends Aborted .	
OUTPUT	Editor's duplicate-id warning didn't block execution. First item's Infrastructure resolution failed as expected, group abort triggered, second item was skipped . Final status showed Failed , not literally "Aborted" — a label discrepancy, not a defect.	

G1	Environment stage-rollback triggers the rollback block	PASS
CHECKING	The scoped rule applies correctly to nodes running inside the rollback path, not just the forward path.	
PREDICTED	Rollback executes; nodes inside it resolve per the same scoped rule as the forward path.	
OUTPUT	Infrastructure failed on the fake infra id, environment-scope stage-rollback fired, rollback step ran and succeeded (Rollback badge). Final pipeline status Failed — expected, since rollback succeeding doesn't retroactively pass the stage.	

G2	Nested-rollback guard, first attempt	INCONCLUSIVE
CHECKING	That a rollback-of-a-rollback is blocked or no-ops, rather than looping or erroring.	
PREDICTED	Second rollback attempt is blocked/no-ops rather than looping.	
OUTPUT	No infinite loop or crash observed — but the authored YAML only exercised <i>one</i> pipeline-rollback trigger, not a genuine "failure while already rolling back" case. Flagged as the riskiest item in the batch; superseded below by a dedicated repro, G2b.	

G2b	Dedicated nested-rollback guard repro	PASS
CHECKING	Directly tests the rollback-of-a-rollback guard: a step <i>inside</i> the <code>rollback:</code> block fails and carries its own step-level <code>on-failure: stage-rollback</code> .	
PREDICTED	Second <code>stage-rollback</code> is blocked/no-op — pipeline ends Failed without looping, hanging, or throwing.	
OUTPUT	Infrastructure failed → environment-scope stage-rollback fired → rollback step that also fails ran (Rollback badge) → failed on exit 1 → its own on-failure: stage-rollback did not spawn a second rollback . No loop, no hang, no engine exception — clean terminal Failed . Guard confirmed solid; resolves G2.	

H1 / H2	Full green regression — unused on-failure + manifest templating	PASS
CHECKING	The render/templating step is unaffected by a present-but-inert service/environment <code>on-failure</code> block.	
PREDICTED	Runs and renders normally; <code>on-failure</code> present but inert since nothing fails.	
OUTPUT	Full green run: Initialize, Service, Infrastructure, both Harness Manifest (templating) steps, Resource Constraint, and real step all Success . Script echoed correctly. Scoped <code>on-failure</code> stayed completely inert.	

H3	Baseline — zero on-failure fields anywhere	PASS
CHECKING	That pipelines never touching this feature at all behave identically to before the change.	
PREDICTED	Identical to before — no regression.	
OUTPUT	Identical green run to H1/H2 with zero on-failure fields present anywhere . No regression.	

I1	Environment scope under kubernetes runtime	PASS (KNOWN LIMITATION)
CHECKING	Whether environment-scope <code>on-failure</code> works under <code>runtime: kubernetes</code> the way it does under <code>shell</code> — a known-limitation check, not a pass/fail gate on the fix itself.	
PREDICTED	Expected to still fail at the Initialize node, not Infrastructure — i.e. expected to <i>not</i> show the fix working.	
OUTPUT	Failed at Initialize, never reached Infrastructure — the environment-scope fix never got a chance to engage under kubernetes. Consistent with the known pre-existing limitation; not a new bug.	

04 Deep dive — G2b's three attempts

G2 only proved a rollback could fire, not that a failure *inside* a rollback avoids recursing. Landing a repro the plan-creator and engine would both accept took three tries:

ATTEMPT 1 – REJECTED AT PLAN CREATION

Service-scope trigger with action: `stage-rollback`.

Invalid request: Failure strategy action [STAGE_ROLLBACK] is not allowed at service scope

ATTEMPT 2 – RAN, BUT PROVED NOTHING

Service-scope trigger with action: `pipeline-rollback`. Ran, but `pipeline-rollback` only rewinds *preceding, already-succeeded* stages — with one stage in the test pipeline, there was nothing before it to roll back. The node came back empty and trivially Success; the injected failing rollback step never executed.

ATTEMPT 3 – THE REAL TEST

Moved the trigger to **environment** scope with action: `stage-rollback` (G1's pattern). This correctly invoked the stage's own `rollback: block`, which failed on `exit 1` and carried its own `on-failure: stage-rollback`. The engine did not spawn a second rollback — it surfaced the plain error and terminated cleanly as **Failed**.